

Binární stromy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

05.06.2026

CC BY-NC-SA 4.0

Binární strom

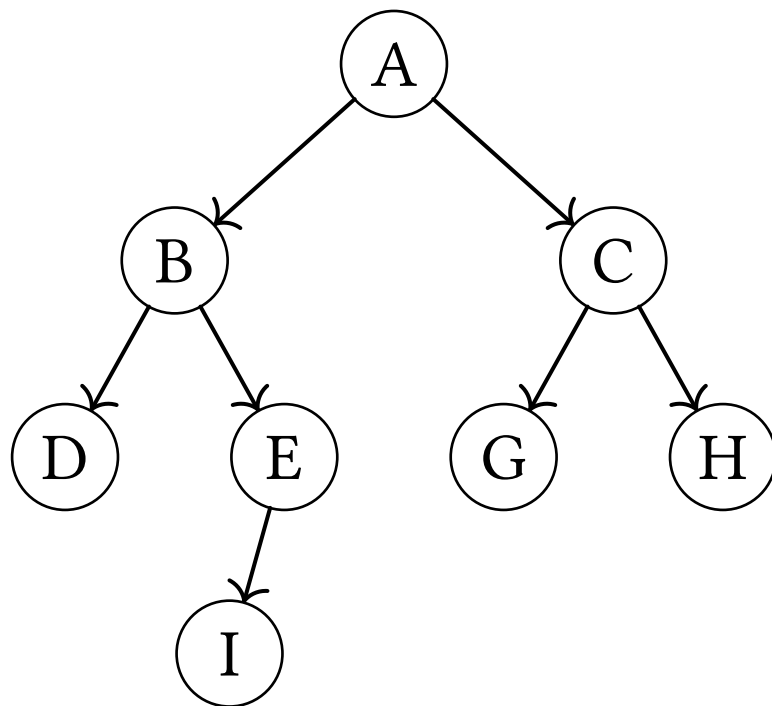
Co je binární strom?

Minule jsme se seznámili s obecnými stromy, kde uzel mohl mít libovolný počet potomků

V praxi se ale nejčastěji setkáváme se stromy, kde má každý uzel **maximálně dva potomky**

Těmto potomkům říkáme **levý** a **pravý**

Příklad binárního stromu



Implementace v C

Obecný strom

U obecného stromu má uzel **libovolný** počet potomků:

```
typedef struct TreeNode {  
    int data;  
    struct TreeNode **children;  
    int child_count;  
} TreeNode;
```

Nevýhodou je, že musíme dynamicky alokovat pole potomků a správa paměti je složitější

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

```
typedef struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Mnohem jednodušší! Podobná struktura jako u spojového seznamu, jen místo jednoho next máme left a right

Binární strom

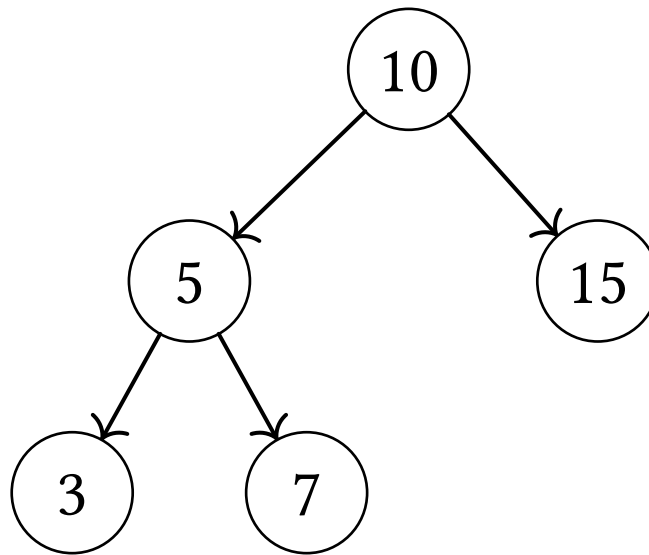
Vytvoření jednoduchého stromu:

```
Node *create_node(int data) {  
    Node *node = (Node *) malloc(sizeof(Node));  
    node->data = data;  
    node->left = NULL; node->right = NULL;  
    return node;  
}
```

```
Node *root = create_node(10);  
root->left = create_node(5);  
root->right = create_node(15);  
root->left->left = create_node(3);  
root->left->right = create_node(7);
```


Binární strom

Výsledek:



Všimněte si, že uzly 15, 3 a 7 mají `left == NULL` a `right == NULL`

Průchody binárním stromem

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý
Preorder	NLR	Uzel , Levý, Pravý
Postorder	LRN	Levý, Pravý, Uzel

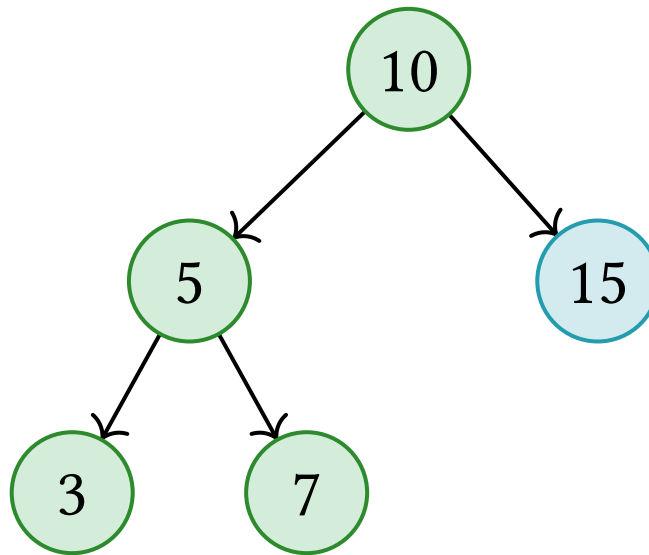
Inorder (LNR)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Zpracuj **aktuální uzel**
3. Rekurzivně projdi **pravý** podstrom

Inorder (LNR)

Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 5, 7, 10, 15

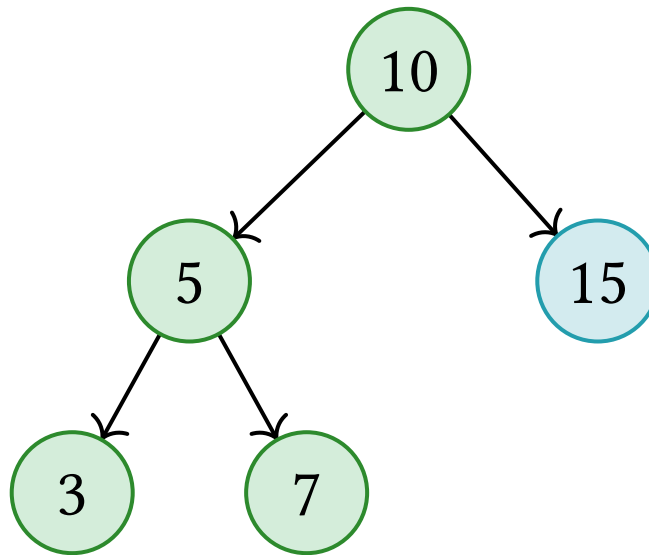
Preorder (NLR)

Postup:

1. Zpracuj **aktuální uzel**
2. Rekurzivně projdi **levý** podstrom
3. Rekurzivně projdi **pravý** podstrom

Preorder (NLR)

Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10, 5, 3, 7, 15

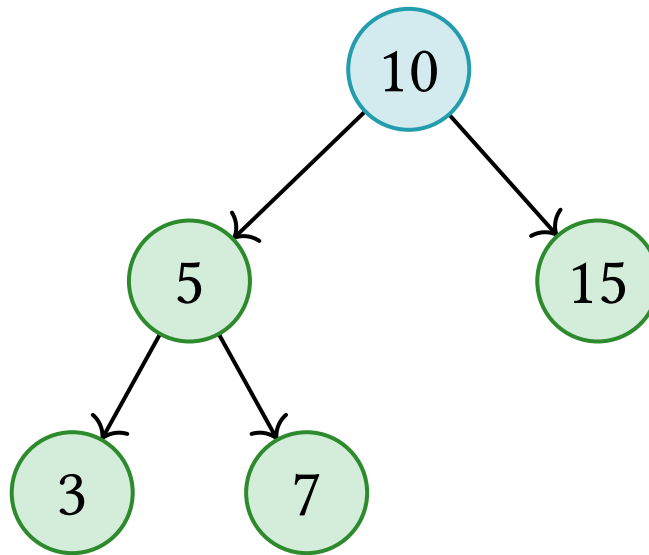
Postorder (LRN)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Rekurzivně projdi **pravý** podstrom
3. Zpracuj **aktuální uzel**

Postorder (LRN)

Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 7, 5, 15, 10

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

- Vyvážený strom: $\mathcal{O}(\log n)$
- Degenerovaný strom (ukážeme později): $\mathcal{O}(n)$

Uvolnění stromu z paměti

Jak byste takto vytvořenou datovou strukturu uvolnili z paměti?

Nemůžeme uvolnit uzel **před** jeho potomky, protože bychom k nim ztratili přístup

```
void free_tree(Node *node) {  
    if (node == NULL) return;  
    free_tree(node->left);  
    free_tree(node->right);  
    free(node);  
}
```

Uvolnění stromu z paměti

Pozor: kdybychom zavolali `free(node)` **před** `free_tree(node->left)`, přistupovali bychom k již uvolněné paměti $\Rightarrow$ **use after free!**

Všimněte si, že `free_tree` vlastně odpovídá **postorder** průchodu – nejprve zpracujeme oba podstromy a teprve pak aktuální uzel

Shrnutí

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- V C implementujeme uzlem se dvěma ukazateli: `left` a `right`
- Tři průchody: **inorder** (LNR), **preorder** (NLR), **postorder** (LRN)
- Všechny průchody mají časovou složitost $\mathcal{O}(n)$
- Paměťová složitost závisí na výšce stromu: $\mathcal{O}(\log n)$ až $\mathcal{O}(n)$